FitForge — Phase 2: Solution Setup, Entities, DbContext, Migrations & Seed Data

Status: **complete and verified on 2026-10-29** (build: 0 warnings, 0 errors; migration applied to LocalDB; seed data checked with SQL queries). No automated tests exist yet (Phase 9).

---

1. What was built

Area	Result
Solution	`FitForge.sln` with 4 projects, SDK pinned by `global.json` (10.0.300, roll forward to newer 10.0 feature bands)
Domain	15 entity classes, 4 enums, `BaseEntity` / `AuditableEntity`, `Roles` constants
Infrastructure	`AppDbContext` (Identity + 13 domain tables), 14 entity configurations, DI registration, startup database initializer, demo data seeder
Database	Migration `InitialCreate` applied to `(localdb)\MSSQLLocalDB` → database `FitForgeDb`
API	`Program.cs` wired up; `GET /health` reports database connectivity

Change from the Phase 1 design

`ApplicationUser` and `ApplicationRole` live in **Domain/Entities**, not Infrastructure/Identity. Domain entities such as `MemberProfile.User` need a navigation property to the user, and the Domain project cannot reference Infrastructure. Domain therefore references the small `Microsoft.Extensions.Identity.Stores` package (no ASP.NET Core or EF Core dependency). JWT/refresh-token services will still live in `Infrastructure/Identity` (Phase 3).

---

2. Packages installed (exact versions)

Project	Package	Version
Domain	Microsoft.Extensions.Identity.Stores	10.0.14
Infrastructure	Microsoft.EntityFrameworkCore.SqlServer	10.0.14
Infrastructure	Microsoft.AspNetCore.Identity.EntityFrameworkCore	10.0.14
Infrastructure	Microsoft.Extensions.Diagnostics.HealthChecks.EntityFrameworkCore	10.0.14
Infrastructure	System.Security.Cryptography.Pkcs	10.0.14 (pins a patched version; SqlClient pulls in 9.0.11, which has published vulnerabilities)
Api	Microsoft.EntityFrameworkCore.Design	10.0.14
Api	Microsoft.AspNetCore.OpenApi	10.0.14 (template shipped 10.0.8, whose Microsoft.OpenApi 2.0.0 dependency has a published vulnerability)

`dotnet restore` reports **no vulnerability warnings** with these versions.

3. Files

```
FitForge/
├─ global.json
├─ FitForge.sln
└─ src/
   ├─ GymManagement.Domain/
   │  ├─ Common/BaseEntity.cs          BaseEntity (int Id), AuditableEntity (CreatedUtc,
UpdatedUtc)
   │  ├─ Common/Roles.cs               "Admin", "Trainer", "Member"
   │  ├─ Enums/Enums.cs                Gender, MembershipStatus, PaymentMethod, BookingStatus
   │  └─ Entities/
   │     ├─ ApplicationUser.cs         ApplicationUser, ApplicationRole (Identity, Guid keys)
   │     ├─ MemberProfile.cs
   │     ├─ TrainerProfile.cs
   │     ├─ Membership.cs              MembershipPlan, Membership (+ IsActiveOn(date))
   │     ├─ Payment.cs
   │     ├─ WorkoutPlan.cs             WorkoutPlan, WorkoutExercise
   │     ├─ DietPlan.cs
   │     ├─ FitnessClass.cs            FitnessClass, ClassBooking, Attendance
   │     └─ Security.cs                RefreshToken, AuditLog
   ├─ GymManagement.Application/       (empty until Phase 4)
   ├─ GymManagement.Infrastructure/
   │  ├─ Configurations/
   │  │  ├─ UserConfigurations.cs      ApplicationUser, MemberProfile, TrainerProfile,
RefreshToken, AuditLog
   │  │  ├─ MembershipConfigurations.cs MembershipPlan, Membership, Payment
   │  │  ├─ TrainingConfigurations.cs  WorkoutPlan, WorkoutExercise, DietPlan
   │  │  └─ ClassConfigurations.cs     FitnessClass, ClassBooking, Attendance
   │  ├─ Data/
   │  │  ├─ AppDbContext.cs
   │  │  ├─ Migrations/                20261029054026_InitialCreate (+ Designer, ModelSnapshot)
   │  │  └─ Seed/
   │  │     ├─ SeedOptions.cs
   │  │     ├─ DatabaseInitializer.cs  migrations + roles + dev admin
   │  │     └─ DemoDataSeeder.cs       dev-only demo data
   │  └─ DependencyInjection.cs        AddInfrastructure(): DbContext, Identity core, health checks
   └─ GymManagement.Api/
      ├─ Program.cs
      ├─ appsettings.json              no secrets, empty connection string
      ├─ appsettings.Development.json  LocalDB connection string, seed switches
      ├─ Properties/launchSettings.json http://localhost:5080, https://localhost:7080
      └─ GymManagement.Api.http        sample requests
```

4. How it works

AppDbContext

- Inherits `IdentityDbContext<ApplicationUser, ApplicationRole, Guid>`, so Identity's tables (`AspNetUsers`, `AspNetRoles`, `AspNetUserRoles`, ...) live in the same database.
- `ApplyConfigurationsFromAssembly` loads every `IEntityTypeConfiguration<T>` class, which keeps `OnModelCreating` short.

- **Delete behaviour:** after the configurations load, every foreign key between domain tables is forced to `Restrict`. Only `WorkoutPlan → WorkoutExercise` and `User → RefreshToken` cascade. So a delete can never silently wipe payments or attendance.
- **UTC handling:** a value converter marks every `DateTime` read from SQL Server as `DateTimeKind.Utc`, so JSON responses end in `Z` and the React app converts them to local time correctly.
- **Automatic timestamps:** `SaveChangesAsync` sets `CreatedUtc` on new rows and `UpdatedUtc` on changed rows. Services never set them by hand.

Constraints enforced by the database (not just C#)

Rule	How
One active booking per member per class, rebooking allowed after cancelling	Filtered unique index `UX_ClassBookings_ActiveBooking … WHERE [Status] = 0`
One attendance per member per class	Filtered unique index `UX_Attendances_MemberPerClass … WHERE [FitnessClassId] IS NOT NULL`
One profile per user	Unique index on `MemberProfiles.UserId` and `TrainerProfiles.UserId`
Unique plan names, unique refresh-token hashes, unique emails	Unique indexes (emails via Identity's `NormalizedEmail` + `RequireUniqueEmail`)
Positive payments, valid dates, sane sets/reps/capacity/calories	`CHECK` constraints (`CK_*`)
Money	`decimal(10,2)`; body measurements `decimal(5,2)`

Startup initializer (`DatabaseInitializer.InitializeDatabaseAsync`)

1. If `Seed:ApplyMigrationsOnStartup` is `true` (development only), runs `Database.MigrateAsync()`.
2. Always creates the three roles if missing.
3. **Development only:** creates the admin from `Seed:AdminEmail` + `Seed:AdminPassword`, then demo data if `Seed:SeedDemoData` is `true`.
4. Every step checks first, so restarting the app does not create duplicates. Demo data is skipped if any membership plan exists.

Demo data (development only, all emails end in `@fitforge.test`)

- 4 plans (Monthly ₹1,500 · Quarterly ₹4,000 · Half-Yearly ₹7,500 · Annual ₹13,000)
- 2 trainers: `arjun.trainer@` (Strength), `priya.trainer@` (Yoga)
- 6 members with memberships relative to today: active, **expiring in ~5 days** (Karthik), **expired** (Vikram), **part-paid** (Divya: ₹4,000 of ₹7,500)
- Classes for the past 7 and next 7 days (Sundays closed), with bookings and past attendance
- 2 workout plans (10 exercises) and 2 diet plans

---

5. Setup commands (what was run)

```
cd C:\Users\ADMIN\projects\FitForge

# Solution and projects
dotnet new globaljson --sdk-version 10.0.300 --roll-forward latestFeature
dotnet new sln -n FitForge --format sln
dotnet new classlib -n GymManagement.Domain -o src/GymManagement.Domain -f net10.0
dotnet new classlib -n GymManagement.Application -o src/GymManagement.Application -f net10.0
dotnet new classlib -n GymManagement.Infrastructure -o src/GymManagement.Infrastructure -f net10.0
dotnet new webapi -n GymManagement.Api -o src/GymManagement.Api -f net10.0 --use-controllers
dotnet sln add (Get-ChildItem -Recurse -Filter *.csproj).FullName

# Project references
dotnet add src/GymManagement.Application reference src/GymManagement.Domain
dotnet add src/GymManagement.Infrastructure reference src/GymManagement.Application
src/GymManagement.Domain
dotnet add src/GymManagement.Api reference src/GymManagement.Application
src/GymManagement.Infrastructure

# Packages
dotnet add src/GymManagement.Domain package Microsoft.Extensions.Identity.Stores --version 10.0.14
dotnet add src/GymManagement.Infrastructure package Microsoft.EntityFrameworkCore.SqlServer --
version 10.0.14
dotnet add src/GymManagement.Infrastructure package
Microsoft.AspNetCore.Identity.EntityFrameworkCore --version 10.0.14
dotnet add src/GymManagement.Infrastructure package
Microsoft.Extensions.Diagnostics.HealthChecks.EntityFrameworkCore --version 10.0.14
dotnet add src/GymManagement.Infrastructure package System.Security.Cryptography.Pkcs --version
10.0.14
dotnet add src/GymManagement.Api package Microsoft.EntityFrameworkCore.Design --version 10.0.14
dotnet add src/GymManagement.Api package Microsoft.AspNetCore.OpenApi --version 10.0.14

# Development secrets (stored in %APPDATA%\Microsoft\UserSecrets, never in the repo)
dotnet user-secrets init --project src/GymManagement.Api
dotnet user-secrets set "Seed:AdminPassword" "<your admin password>" --project
src/GymManagement.Api
dotnet user-secrets set "Seed:DemoPassword"  "<demo accounts password>" --project
src/GymManagement.Api

# Migration
dotnet ef migrations add InitialCreate --project src/GymManagement.Infrastructure --startup-project
src/GymManagement.Api --output-dir Data/Migrations
```

Passwords must have 8+ characters with an uppercase letter, a lowercase letter, a digit and a symbol, or seeding stops with a clear error.

---

6. Run and verify

```
dotnet build FitForge.sln
dotnet run --project src/GymManagement.Api --launch-profile http
```

Expected console output on first run:

```
info: DatabaseInitializer[0]  Applying database migrations
info: DatabaseInitializer[0]  Created role Admin / Trainer / Member
info: DatabaseInitializer[0]  Created development admin admin@fitforge.test
info: DatabaseInitializer[0]  Seeded demo data: 2 trainers, 6 members, 4 plans, classes for ±7 days
      Now listening on: http://localhost:5080
```

Then:

- `GET http://localhost:5080/health` → `Healthy`
- SSMS → connect to `(localdb)\MSSQLLocalDB` → database `FitForgeDb`

Row counts verified after seeding:

Table	Rows	Table	Rows
AspNetUsers	9	FitnessClasses	24
AspNetRoles	3	ClassBookings	72
MemberProfiles	6	Attendances	36
TrainerProfiles	2	WorkoutPlans	2
MembershipPlans	4	WorkoutExercises	10
Memberships	6	DietPlans	2
Payments	6	__EFMigrationsHistory	1

Constraint checks run against the database (each in a rolled-back transaction):

Attempt	Result
Duplicate active booking	Rejected by `UX_ClassBookings_ActiveBooking`
Cancel, then rebook the same class	Accepted (intended)
Negative payment	Rejected by `CK_Payments_Amount`
Class ending before it starts	Rejected by `CK_FitnessClasses_Times`
Delete a plan that has memberships	Rejected by the `Restrict` foreign key

Useful SQL in SSMS:

```
SELECT u.Email, r.Name AS Role
FROM AspNetUsers u
JOIN AspNetUserRoles ur ON ur.UserId = u.Id
JOIN AspNetRoles r ON r.Id = ur.RoleId
ORDER BY r.Name, u.Email;
```

To start over with a fresh database:

```
dotnet ef database drop --project src/GymManagement.Infrastructure --startup-project
src/GymManagement.Api --force
dotnet run --project src/GymManagement.Api --launch-profile http
```

---

7. Common mistakes

- **Running with `--no-build` right after adding a migration.** The old DLL has no migrations, so EF creates an empty database and seeding fails with `Invalid object name 'AspNetRoles'`. Build first. (This happened once during Phase 2. After a rebuild, the next run applied the migration to the same database; no drop was needed.)
- **Missing `--startup-project`** in `dotnet ef` commands. The migrations live in Infrastructure, but configuration (connection string, user secrets) lives in Api.
- **Weak seed passwords.** Identity's password rules apply to seeded accounts too.
- **Putting passwords in `appsettings.json`.** Use user secrets locally and environment variables (`Seed__AdminPassword`) on servers. Outside development the admin and demo data are never seeded.
- **Hard-deleting users.** Set `IsActive = false` instead; the `Restrict` foreign keys will block deletes that would orphan history anyway.
- **Using `DateTime.Now`.** Always `DateTime.UtcNow`; the gym's local time (IST) is only for display.

8. Security notes for this phase

- No secrets are committed: the connection string uses Windows authentication and passwords are in user secrets.
- Passwords are hashed by Identity (PBKDF2-HMAC-SHA512, 100,000 iterations, per-user salt). Hashes start with `AQAAAAIAAYag`.
- Account lockout is configured now (5 failures → 15 minutes) and takes effect when login is built in Phase 3.
- The seeder logs emails only, never passwords.

Phase checklist

- [x] Phase 1 — Architecture, requirements, data model, folder structure, dependencies
- [x] Phase 2 — Solution setup, entities, DbContext, configurations, migrations, seed data
- [ ] Phase 3 — Identity, password hashing, JWT, refresh tokens, roles, policies
- [ ] Phase 4 — Services, DTOs, validation, CRUD APIs, resource-level authorization
- [ ] Phase 5 — Rate limiting, CORS, exception handling, audit logging, Swagger, hardening
- [ ] Phase 6 — React setup, design system, routing, auth integration, layouts
- [ ] Phase 7 — Admin, trainer, member dashboards on live APIs
- [ ] Phase 8 — Memberships, payments, workouts, diets, bookings, attendance UI
- [ ] Phase 9 — Automated tests, fixes, responsive and security checks
- [ ] Phase 10 — README, demo prep, resume description